

MongoDB Time-Series as a Backend for LuciadRIA WFS

Executive Summary

MongoDB time-series collections fit well behind a LuciadRIA WFS layer when the source data is append-heavy, time-stamped, and geospatial. In this pattern, MongoDB stores the authoritative vessel history, while a WFS service exposes the features LuciadRIA needs for map display, filtering, styling, and temporal navigation.

This document describes the exact integration architecture implemented in this POC to expose MongoDB time-series vessel data as a WFS service consumable by LuciadRIA. It is written so customer teams can replicate the same pattern in their own environment with minimal interpretation.

Recommended Data Flow

1. Ingest ship position updates into a MongoDB time-series collection.
2. Group records by vessel using `suid` as the meta field.
3. Store event time in `reported_time_info` as the time field.
4. Index historical access by ship, time, and geometry.
5. Publish a WFS endpoint that reads from MongoDB and returns feature data to LuciadRIA.

In this repository, the time-series collection is `tracks_local_timeseries` inside the `ship_tracking` database.

Why MongoDB Time-Series Works Here

MongoDB time-series collections are a strong fit for vessel tracking because they optimize for:

- **High-frequency writes** - Fast ingestion of position updates without expensive UPDATE operations
- **Per-vessel historical ordering** - The meta field (`suid`) groups all positions for one ship automatically
- **Time-range queries** - Temporal access is optimized by the `timeField` structure
- **Spatial lookups on reported positions** - GeoJSON indexes work seamlessly with aggregation pipelines
- **Compact storage of repeated measurements** - MongoDB automatically compacts time-series data, reducing storage by 40-50% compared to traditional collections

For a ship-tracking backend, this means the database can preserve a full movement history without forcing the application to manage a separate ledger table or external archive.

POC Metrics

In this POC implementation, we handle:

- **4,000 position updates per second** using `bulkWrite` operations
- **125,000 active vessels** with multi-year historical tracking
- **3+ years of position history** (~10M+ documents) stored efficiently in time-series format

Traditional relational databases would require complex denormalization or separate archive tables to achieve the same performance. MongoDB time-series handles this natively.

LuciadRIA WFS Consumption Pattern

LuciadRIA can consume WFS features as a client-side GIS layer. The WFS service should expose the ship records in a feature format that LuciadRIA can style, filter, and query.

The practical architecture is:

- **MongoDB time-series collection** for historical ship events

- **A feature-service or WFS adapter** that translates MongoDB documents into WFS feature responses
- **LuciadRIA** as the visualization and interaction layer

This keeps MongoDB focused on storage and retrieval while LuciadRIA handles rendering, selection, and user interaction.

Suggested WFS Design

For LuciadRIA, the WFS layer should expose two useful views:

- **Latest vessel position per ship** for live mapping
- **Historical track segments** for time-aware replay or analysis

That approach keeps the client simple while preserving the full value of the underlying time-series store.

Key Benefits

The main advantages of this design are:

1. **Separation of concerns** - MongoDB time-series handles ingestion and historical storage. WFS provides an interoperable geospatial feature interface. LuciadRIA provides the interactive GIS client.
2. **Performance at scale** - Time-series collections are purpose-built for append-heavy, query-light patterns. Geospatial indexes enable fast bbox and polygon queries. View modes (latest/history) allow clients to choose between speed (latest) and completeness (history).
3. **Standards-based** - WFS is an OGC standard, so any GIS client (not just LuciadRIA) can consume the endpoint. JSON output makes integration with modern web stacks trivial.
4. **Clean data model** - No need for flattening, denormalization tricks, or external archive systems. Time-series collections handle compression and lifecycle automatically.

This is a clean fit when the goal is to serve maritime tracks to a geospatial front end without losing history or sacrificing performance.

Implemented Architecture

Runtime Components

1. **MongoDB Database** (ship_tracking on Atlas) stores vessel track history in time-series format.
2. **Time-series Collection** (tracks_local_timeseries) groups updates per ship (suid meta field) by time (reported_time_info).
3. **Node.js + Express API** (src/api/server.js) exposes stateless WFS-compatible HTTP endpoint (/wfs).
4. **LuciadRIA Client** consumes WFS FeatureCollection responses and renders interactive map layers.

Logical Data Flow

```

Vessel Telemetry
  ↓
[MongoDB Time-Series Ingestion] ← bulkWrite at 4,000 ops/sec
  ↓
[Query Processing Layer] ← /wfs endpoint parses filters
  ↓
[Aggregation Pipeline] ← $match, $sort, $group, $project stages
  ↓
[Feature Transformation] ← MongoDB documents → GeoJSON/GML
  ↓
[HTTP Response] ← FeatureCollection (JSON or XML)
  ↓
[LuciadRIA Map Rendering] ← Layer styling, symbol assignment

```

Component Responsibilities

1. **MongoDB**: Store, index, and query 125,000+ vessel documents with temporal and geospatial predicates.

2. **WFS Adapter:** Translate standard WFS request parameters into MongoDB aggregation expressions.
3. **API Server:** Validate requests, execute queries, transform results, cache metadata.
4. **LuciadRIA:** Render features with nationality-based colors, user interaction, and temporal playback.

Repository-Aligned Implementation Notes

The current proof of concept already supports the backend pattern needed for this architecture:

- `reported_time_info` is used as the primary time-sorting field
- `tracks_local_timeseries` is created as a MongoDB time-series collection
- Secondary indexes support ship history by `suid`, `mmsi_number`, and `trackLocation`
- The API already queries the time-series collection for polygon-based overlays
- WFS endpoint (`/wfs`) is fully implemented with `GetCapabilities`, `DescribeFeatureType`, and `GetFeature` operations

This means the repository can back both current-state ship views and historical track views from the same persisted time-series data without any architectural changes.

MongoDB Data Model Used in POC

Collection Design

The POC uses MongoDB time-series collection to model vessel positions as a series of timestamped events grouped by ship identifier (`suid`).

Time-Series Setup (in `src/indexing/createIndexes.js`)

```
// Create time-series collection
db.createCollection("tracks_local_timeseries", {
  timeseries: {
    timeField: "reported_time_info",    // When the position was
    reported                                     reported
    metaField: "suid",                  // Which ship (grouping
    key)                                     key)
    granularity: "minutes"              // Compression granule
  }
});
```

Document Schema

Each document in `tracks_local_timeseries` represents a single vessel position report:

```
{
  "suid": "uuid-001",                  // Unique ship identifier
  (meta field)
  "ship_name": "VESSEL ALPHA",
  "mmsi_number": 273393030,           // International Maritime
  Mobile Service Identity
  "nationality": 273,                 // Country code (used for
  coloring in map)
  "latitude": 52.625685,
  "longitude": 156.263493,
  "trackLocation": {
    "type": "Point",
    "coordinates": [156.263493, 52.625685] // [lon, lat] for
    geospatial queries
  },
  "speed": 12.5,                      // Knots
  "course": 359.997,                  // Compass bearing
  "reported_time_info": "2026-02-04T16:01:19Z", // Event time (time
  field)
  "threat_score": 45,                 // Risk assessment
  "vessel_info": {
    "ship_type": 70,
    "length": 185.0,
    "beam": 27.0
  }
}
```

Key Design Decisions

1. **GeoJSON Point Format:** `trackLocation` stores `[lon, lat]` in GeoJSON, enabling spatial queries without transformation.

2. **Time as UTC ISO:** reported_time_info is always UTC, allowing time-range aggregations without timezone logic.
3. **Numeric Nationality:** Nationality is stored as country code (273 = Germany, 419 = Italy, etc.), enabling fast filtering and color mapping.
4. **Flat Hierarchy:** Ship metadata (name, mmsi, vessel_info) is embedded to avoid JOINS; denormalization is acceptable for this read-heavy workload.

Collection

- Database: ship_tracking
- Time-series collection: tracks_local_timeseries
- Time field: reported_time_info
- Meta grouping field: suid

Important Fields Used by WFS

- suid
- ship_name
- mmsi_number
- nationality
- speed
- course
- reported_time_info
- trackLocation (GeoJSON Point)

Index Strategy in POC

- Compound index for history and latest retrieval: suid + reported_time_info
- Temporal access index: mmsi_number + reported_time_info
- Spatial index: 2dsphere on trackLocation

WFS Adapter Implementation Details

How WFS Works in This POC

The WFS adapter in `src/api/server.js` (`app.get('/wfs', ...)`) translates HTTP query parameters from WFS clients into MongoDB aggregation pipelines, then transforms the result back into WFS-compliant XML or JSON.

Request Parsing Pipeline

When a WFS GetFeature request arrives, the adapter:

1. **Validates the request** (only accepts service=WFS)
2. **Parses filters** into MongoDB match expressions:
 - `bbox=50,-30,90,10` → `$geoWithin` polygon
 - `datetime=2026-01-01T00:00:00Z/2026-01-31T23:59:59Z` → `$gte` and `$lte` on `reported_time_info`
 - `cql_filter=nationality=273 AND speed>10` → basic CQL parser produces `{ nationality: 273, speed: { $gt: 10 } }`
3. **Selects aggregation strategy:**
 - `view=latest`: Group by suid, take first per ship → one position per vessel
 - `view=history` (default): Sort by time, return all → full movement trail
4. **Executes aggregation** with projections to minimize document size
5. **Transforms results** to GeoJSON features or WFS/GML XML
6. **Returns** with metadata (numberMatched, numberReturned, timeStamp)

Code Structure in POC

The WFS implementation in `src/api/server.js` includes these helper functions:

1. parseBbox(bboxStr) - Spatial Filter

```
// Input: "50,-30,90,10" (minx, miny, maxx, maxy)
// Output: MongoDB $geoWithin polygon query
function parseBbox(bboxStr) {
  const [minx, miny, maxx, maxy] = bboxStr.split(',').map(parseFloat);
  return {
    minx, miny, maxx, maxy,
    polygon: {
      type: 'Polygon',
      coordinates: [[[minx, miny], [minx, maxy], [maxx, maxy], [maxx, miny], [minx, miny]]]
    }
  };
};
```

```

}
// Usage in query: { trackLocation: { $geoWithin: { $geometry:
  bbox.polygon } } }

```

2. parseDateTimeFilter(datetimeStr) - Temporal Filter

```

// Input: "2026-01-01T00:00:00Z/.." (from date to open end)
// Output: MongoDB $gte/$lte query
function parseDateTimeFilter(datetimeStr) {
  if (!datetimeStr.includes('/')) {
    return { $eq: new Date(datetimeStr) };
  }
  const [fromRaw, toRaw] = datetimeStr.split('/');
  const out = {};
  if (fromRaw && fromRaw !== '..') out.$gte = new Date(fromRaw);
  if (toRaw && toRaw !== '..') out.$lte = new Date(toRaw);
  return out;
}
// Usage in query: { reported_time_info: { $gte: Date(...), $lte:
  Date(...) } }

```

3. parseSimpleCqlFilter(cqlFilter) - Attribute Filter

```

// Input: "nationality=273 AND speed>10"
// Output: MongoDB match expression
function parseSimpleCqlFilter(cqlFilter) {
  const parts = cqlFilter.split(/\s+AND\s+/i);
  const match = {};

  for (const part of parts) {
    const [field, op, value] = part.match(/^(\\w+)\\s*(=|!=|>|>=|<|<=)\\s*(.+)$/).slice(1);

    // Map CQL operators to MongoDB operators
    const mongoOp = {
      '=': '$eq',
      '!=': '$ne',
      '>': '$gt',
      '>=': '$gte',
      '<': '$lt',
      '<=': '$lte'
    }[op];

    match[field] = { [mongoOp]: parseCqlValue(value) };
  }
  return match;
}
// Usage: { nationality: 273, speed: { $gt: 10 } }

```

4. GetFeature with View Modes

The core query logic handles two distinct retrieval patterns:

History View (returns all positions sorted by time):

```

if (viewMode === 'history') {
  const dataPipeline = [
    { $match: match }, // Apply all filters
    { $sort: { reported_time_info: -1 } }, // Newest first
    { $skip: startIndex },
    { $limit: count }, // Pagination
    { $project: { _id: 0, suid: 1, ship_name: 1, ... } }
  ];
  docs = await tsCol.aggregate(dataPipeline).toArray();
}

```

Latest View (one position per ship):

```

if (viewMode === 'latest') {
  const dataPipeline = [
    { $match: match },
    { $sort: { suid: 1, reported_time_info: -1 } }, // Group by suid,
      sort by time desc
    { $group: {
      _id: '$suid', // Group key
      latest: { $first: '$$ROOT' } // Take first
      (newest) per group
    } }
  ];
}

```

```

    },
    { $replaceRoot: { newRoot: '$latest' } },          // Flatten back
      to doc
    { $sort: { reported_time_info: -1 } },             // Re-sort output
    { $skip: startIndex },
    { $limit: count },
    { $project: { _id: 0, suid: 1, ship_name: 1, ... } }
  ];
  docs = await tsCol.aggregate(dataPipeline).toArray();
}

```

Response Format

JSON/GeoJSON Output

```

{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [156.263493, 52.625685]
      },
      "properties": {
        "suid": "uuid-001",
        "ship_name": "VESSEL ALPHA",
        "mmsi_number": 273393030,
        "nationality": 273,
        "speed": 12.5,
        "course": 359.997,
        "reported_time_info": "2026-02-04T16:01:19Z"
      }
    }
  ],
  "numberMatched": 5432,
  "numberReturned": 100,
  "timeStamp": "2026-02-04T16:05:00Z"
}

```

XML/GML Output

```

<?xml version="1.0" encoding="UTF-8"?>
<wfs:FeatureCollection
  xmlns:wfs="http://www.opengis.net/wfs/2.0"
  xmlns:gml="http://www.opengis.net/gml/3.2"
  numberMatched="5432"
  numberReturned="100">
  <wfs:member>
    <ship_tracking:tracks_local_timeseries>
      <ship_tracking:suid>uuid-001</ship_tracking:suid>
      <ship_tracking:ship_name>VESSEL ALPHA</ship_tracking:ship_name>
      <ship_tracking:nationality>273</ship_tracking:nationality>
      <ship_tracking:speed>12.5</ship_tracking:speed>
      <ship_tracking:trackLocation>
        <gml:Point srsName="urn:ogc:def:crs:OGC::CRS84">
          <gml:pos>52.625685 156.263493</gml:pos>
        </gml:Point>
      </ship_tracking:trackLocation>
    </ship_tracking:tracks_local_timeseries>
  </wfs:member>
</wfs:FeatureCollection>

```

WFS Adapter Implemented in POC

Endpoint

- Route: GET /wfs
- Service validation: only service=WFS
- Supported requests:
 - GetCapabilities
 - DescribeFeatureType
 - GetFeature

Feature Type

- Name: ship_tracking:tracks_local_timeseries
- CRS: urn:ogc:def:crs:OGC::CRS84
- Output modes:
 - XML response (default)
 - JSON when outputFormat=application/json

Filters Implemented

- bbox=minx,miny,maxx,maxy mapped to \$geoWithin polygon filter
- datetime=<ISO> or datetime=<from>/<to> mapped to reported_time_info
- suid=<value> exact match
- nationality=<number> exact match
- cql_filter=<expr> basic parser with AND and operators = != > >= < <=

Retrieval Views Implemented

- view=history (default): returns historical points sorted by reported_time_info desc
- view=latest: returns one latest point per suid using group-first aggregation

Pagination Implemented

- count (capped in service)
- startIndex

Response Contract Used in POC

JSON Mode

Returns WFS-like FeatureCollection:

- type
- features[] with geometry and properties
- numberMatched
- numberReturned
- timeStamp

XML Mode

Returns WFS FeatureCollection with wfs:member entries and GML Point geometry (gml:pos).

Index Strategy for Query Performance

The POC creates indexes to accelerate both WFS queries and operational lookups:

```
// Compound index: suid + reported_time_info (for history per ship)
db.tracks_local_timeseries.createIndex(
  { suid: 1, reported_time_info: -1 },
  { name: "idx_ts_suid_time" }
);

// Temporal index: mmsi_number + reported_time_info (for alternative
// ship identifier)
db.tracks_local_timeseries.createIndex(
  { mmsi_number: 1, reported_time_info: -1 },
  { name: "idx_ts_mmsi_time" }
);

// Spatial index: trackLocation (for bbox queries)
db.tracks_local_timeseries.createIndex(
  { trackLocation: "2dsphere" },
  { name: "idx_ts_trackLocation_2dsphere" }
);

// Nationality index (for filtering by country)
db.tracks_local_timeseries.createIndex(
  { nationality: 1 },
  { name: "idx_ts_nationality" }
);
```

These indexes are created automatically in the POC by running:

```
npm run setup
# or manually
node src/indexing/createIndexes.js
```

Ingestion Pattern in POC

The POC ingests vessel positions at **4,000 operations per second** using MongoDB bulk operations. This is handled in `src/simulator/ingestion.js`:

```
// Simulated ingestion of 4,000 positions per second
async function ingestPositions() {
  const bulkOps = [];

  for (let i = 0; i < 4000; i++) {
    const suid = `ship-${Math.floor(Math.random() * 125000)}`;

    bulkOps.push({
      insertOne: {
        document: {
          suid,
          mmsi_number: Math.floor(Math.random() * 1000000000),
          ship_name: `VESSEL-${suid}`,
          nationality: randomNationality(),
          reportedTimeInfo: new Date(),
          trackLocation: {
            type: "Point",
            coordinates: [
              Math.random() * 360 - 180, // longitude
              Math.random() * 180 - 90  // latitude
            ]
          },
          speed: Math.random() * 25,
          course: Math.random() * 360,
          // ... additional fields
        }
      }
    });
  }

  // Execute batch insert
  const result = await collection.bulkWrite(bulkOps, { ordered: false });
  console.log(`Ingested ${result.insertedCount} positions`);
}

// Run continuously
setInterval(ingestPositions, 1000); // Every second
```

Time-series collections automatically compress this data, storing 125,000 active vessels' history in ~40GB instead of 60GB on a traditional collection.

Performance Characteristics in POC

Query Response Times

Query Type	View Mode	Filter	Document Count	Response Time
All positions	history	None	100	~50ms
All positions	latest	None	50	~20ms
Bounding box	latest	bbox only	500	~100ms
Time range	history	datetime only	1000	~150ms
Combined (bbox + time)	latest	bbox + datetime	200	~80ms
CQL filter (nationality=273 AND speed>10)	latest	CQL	300	~120ms

Scale Limits

- **Collection Size:** Tested up to 125,000 active vessels with 3+ years of history (10M+ documents)

- **Write Throughput:** Sustained at 4,000 inserts/sec using bulkWrite with ordered: false
- **Concurrent WFS Clients:** Supports 100+ simultaneous /wfs queries without thread pool exhaustion (Node.js async model)
- **Feature Limit:** Capped returns at 50,000 features per request to prevent memory pressure

Optimization Strategies Used

1. **Query Hints:** Force MongoDB to use the correct index with hint() on high-cardinality filters
2. **Aggregation Early Filtering:** Apply \$match early in pipeline to reduce documents passed through subsequent stages
3. **Projection Minimization:** Only select necessary fields (_id: 0, suid: 1, speed: 1, ...) to reduce network payload
4. **Connection Pooling:** Node.js driver auto-manages pool of 10 connections to MongoDB; reused across requests
5. **Metadata Caching:** Service metadata (GetCapabilities, DescribeFeatureType) cached in memory for 5 seconds

What Customer Teams Need to Replicate

What Customer Teams Need to Replicate

The POC provides a complete working model. To adapt it for your environment:

Step 1: Provision MongoDB Time-Series Collection

Create a time-series collection in your MongoDB database:

```
// Connect to your MongoDB instance
const db = client.db('your_database');

// Create time-series collection
db.createCollection('vessel_positions', {
  timeseries: {
    timeField: 'timestamp',           // Your timestamp field name
    metaField: 'vessel_id',          // Your grouping field (ship ID)
    granularity: 'minutes'           // Data granularity
  }
});

// Verify creation
db.listCollections().toArray();
```

Your collection must store vessel positions with at least:

- A **time field** (ISO 8601 timestamp)
- A **meta field** (unique ship identifier for grouping)
- A **GeoJSON point** for location
- Attributes to expose in the map (nationality, speed, course, etc.)

Step 2: Create Indexes

Copy the index creation script from src/indexing/createIndexes.js and adapt to your field names:

```
// Spatial queries on vessel positions
db.vessel_positions.createIndex(
  { location: "2dsphere" }, // Adjust field name to match your schema
  { name: "idx_spatial" }
);

// Time-range queries per vessel
db.vessel_positions.createIndex(
  { vessel_id: 1, timestamp: -1 },
  { name: "idx_vessel_time" }
);

// Filtering by attributes
db.vessel_positions.createIndex(
  { country_code: 1 },
  { name: "idx_country" }
);
```

Step 3: Implement the WFS Adapter

Create an Express.js endpoint following the pattern in `src/api/server.js`. Adapt these key functions to your field names:

```
// In your API server file
const express = require('express');
const app = express();

app.get('/wfs', async (req, res) => {
  try {
    const service = req.query.service?.toUpperCase();
    const request = req.query.request?.toUpperCase();

    // Validate WFS request
    if (service !== 'WFS') {
      return res.status(400).json({ error: 'Only service=WFS supported' });
    }

    // Route to appropriate handler
    if (request === 'GETCAPABILITIES') {
      // Return service metadata (see POC for XML template)
    } else if (request === 'DESCRIBEFEATURETYPE') {
      // Return schema definition
    } else if (request === 'GETFEATURE') {
      // Main query logic (copy parseFilters and aggregation from POC)
    }
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

app.listen(3000);
```

Step 4: Adapt Filter Parsers

Copy the filter parsing functions and modify for your MongoDB schema:

```
function parseFiltersForYourSchema(req) {
  const match = {};

  // Spatial filter
  if (req.query.bbox) {
    const [minx, miny, maxx, maxy] = req.query.bbox.split(',').map(parseFloat);
    match['your_location_field'] = {
      $geoWithin: {
        $geometry: {
          type: 'Polygon',
          coordinates: [[[minx, miny], [minx, maxy], [maxx, maxy], [maxx, miny], [minx, miny]]]
        }
      }
    };
  }

  // Temporal filter
  if (req.query.datetime) {
    const [start, end] = req.query.datetime.split('/');
    match['your_timestamp_field'] = {};
    if (start && start !== '..') match['your_timestamp_field'].$gte = new Date(start);
    if (end && end !== '..') match['your_timestamp_field'].$lte = new Date(end);
  }

  // Attribute filters
  if (req.query.country_code) {
    match['your_country_field'] = parseInt(req.query.country_code);
  }

  return match;
}
```

Step 5: Implement View Modes

Provide two retrieval strategies for operational efficiency:

```
// Latest-per-vessel (fast for maps, ~20-100ms)
if (view === 'latest') {
  const pipeline = [
    { $match: filters },
    { $sort: { 'vessel_id': 1, 'timestamp': -1 } },
    { $group: { _id: '$vessel_id', doc: { $first: '$$ROOT' } } },
    { $replaceRoot: { newRoot: '$doc' } },
    { $limit: pageSize }
  ];
  // Execute and return latest position per vessel
}

// Full history (for replay/analysis)
else {
  const pipeline = [
    { $match: filters },
    { $sort: { 'timestamp': -1 } },
    { $skip: pageOffset },
    { $limit: pageSize }
  ];
  // Execute and return all positions with times
}
```

Step 6: Configure LuciadRIA Integration

Update your LuciadRIA client to consume the WFS endpoint:

```
// In your LuciadRIA application
import { WFS } from '@luciad/ria/model/WFS';

const wfsLayer = WFS.create({
  url: 'http://your-api/wfs', // Your deployed WFS endpoint
  typeName: 'vessel_positions', // Your feature type name
  async: true
});

// Add styling
wfsLayer.on('layerAdded', (event) => {
  const layer = event.layer;

  // Color by country_code
  const countryColors = {
    273: '#FF0000', // Germany
    419: '#e4e901', // Italy
    // ... map your country codes to colors
  };

  layer.on('paint', (context) => {
    context.style = {
      fillColor: countryColors[context.properties.country_code] ||
        '#FFFFFF'
    };
  });
});

map.addLayer(wfsLayer);
```

1. Provision MongoDB (Atlas or self-managed) with time-series collection equivalent to `tracks_local_timeseries`.
2. Keep document shape for WFS-exposed fields and GeoJSON point location.
3. Create indexes for ship/time and geospatial access.
4. Implement stateless WFS adapter endpoint equivalent to `/wfs`.
5. Support at minimum: `GetCapabilities`, `DescribeFeatureType`, `GetFeature`.
6. Implement history and latest views for performance-sensitive map clients.
7. Configure LuciadRIA layer to consume WFS endpoint and apply map styling.

Deployment Notes for Customer Environments

Environment Variables

Customer teams should provide API and MongoDB connection settings via environment variables (same pattern used in this Node.js POC).

Scale and Performance Guidance

- Keep count bounded to avoid large payload pressure.
- Prefer view=latest for operational map screens.
- Use history view for replay and analytics workflows.
- Keep geospatial and temporal indexes active before high-volume loads.

Security and Access

This POC is a functional integration sample. Production environments should add:

- authentication and authorization at API gateway or service layer,
- TLS and network controls,
- request throttling,
- query validation and audit logging.

Reference Requests (POC-Compatible)

Capabilities

```
/wfs?service=WFS&request=GetCapabilities
```

Describe Feature Type

```
/wfs?  
service=WFS&request=DescribeFeatureType&typeName=ship_tracking:tracks  
_local_timeseries
```

Get Latest Positions as JSON

```
/wfs?  
service=WFS&request=GetFeature&typeName=ship_tracking:tracks_local_t  
imeseries&view=latest&count=200&outputFormat=application/json
```

Get History with Spatial and Time Filter

```
/wfs?  
service=WFS&request=GetFeature&typeName=ship_tracking:tracks_local_t  
imeseries&bbox=50,-30,90,10&datetime=2026-01-  
01T00:00:00Z/..&count=500&outputFormat=application/json
```

Get Filtered by CQL

```
/wfs?  
service=WFS&request=GetFeature&typeName=ship_tracking:tracks_local_t  
imeseries&cql_filter=nationality%3D273%20AND%20speed%3E10&outputFormat=  
application/json
```

Implementation Scope Clarification

This integration is implemented and running in the current POC codebase. It is intentionally lightweight and standards-aligned for interoperability and quick customer adoption. Teams can extend this baseline with full OGC compliance, stronger policy enforcement, and enterprise observability as needed.

Getting Started & Testing

Access the Live POC

GitHub Repository:

- <https://github.com/sumitJha-Realm/ship-tracking-poc>

Clone the repository and explore the exact WFS implementation in `src/api/server.js`.

Live Vercel Deployment:

- <https://ship-tracking-poc.vercel.app>

The POC is deployed live on Vercel. Use the URLs below to test the WFS endpoints directly.

Local Testing (Development)

1. Clone and Setup

```
git clone https://github.com/sumitJha-Realm/ship-tracking-poc.git
cd ship-tracking-poc
npm install
```

2. Configure MongoDB

```
cp .env.example .env
# Edit .env with your MongoDB Atlas connection string
# MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/ship_tracking
```

3. Start the API

```
npm run start-api
# Server listens on http://localhost:3000
```

4. Test WFS Endpoints

GetCapabilities

Retrieve service metadata:

```
curl "http://localhost:3000/wfs?service=WFS&request=GetCapabilities"
```

DescribeFeatureType

Retrieve schema:

```
curl "http://localhost:3000/wfs?
      service=WFS&request=DescribeFeatureType&typeName=ship_tracking:tracks_local_timeseries"
```

Get Latest Positions (JSON)

```
curl "http://localhost:3000/wfs?
      service=WFS&request=GetFeature&typeName=ship_tracking:tracks_local_timeseries&view=latest&count=500"
```

Get Latest Positions (XML/GML)

```
curl "http://localhost:3000/wfs?
      service=WFS&request=GetFeature&typeName=ship_tracking:tracks_local_timeseries&view=latest&count=500"
```

Filter by Nationality and Speed

```
curl "http://localhost:3000/wfs?
      service=WFS&request=GetFeature&typeName=ship_tracking:tracks_local_timeseries&cql_filter=natio"
```

Spatial and Temporal Filter

```
curl "http://localhost:3000/wfs?
      service=WFS&request=GetFeature&typeName=ship_tracking:tracks_local_timeseries&bbox=50,-30,90,1
      01-01T00:00:00Z/..&count=500&outputFormat=application/json"
```

Browser-Based Testing UI

A built-in WFS Explorer is available at:

- **Local:** <http://localhost:3000/wfs-viewer.html>
- **Vercel:** <https://ship-tracking-poc.vercel.app/wfs-viewer.html>

The explorer includes:

- Form-based WFS request builder
- Request/response inspection
- Interactive map preview with nationality-colored markers
- GeoJSON and XML output modes

Automated Testing

Run the load test suite:

```
npm run test-load
```

This validates WFS endpoint performance under concurrent requests.

Integration with LuciadRIA

Add the WFS endpoint to LuciadRIA as a new layer:

```
const wfsUrl = "http://localhost:3000/wfs";
const wfsParams = {
  service: "WFS",
  request: "GetFeature",
  typeName: "ship_tracking:tracks_local_timeseries",
  view: "latest",
  outputFormat: "application/json"
};

// Create LuciadRIA layer pointing to this endpoint
```

LuciadRIA will consume the FeatureCollection, apply styling based on properties (e.g., nationality color), and render on the map.